

Laboratorium 6

Filip Piskorski

Informatyka - DS3, 2 stopień, 2 semestr, 2025/2026

1. Program

Program ma 2 części. Pierwsza z nich Sprawdza wykonanie dla różnych wielkości wektorów. Druga sprawdza czas wykonania dla różnego rozdziału wektora na bloki:

```
118 int main(void) {
119     constexpr std::array<std::pair<int, int>, 16> testCases = {
120         std::make_pair(16, 16),
121         std::make_pair(64, 16),
122         std::make_pair(256, 16),
123         std::make_pair(256, 32),
124         std::make_pair(256, 64),
125         std::make_pair(1024, 64),
126         std::make_pair(4096, 64),
127         std::make_pair(4096, 128),
128         std::make_pair(4096, 256),
129         std::make_pair(16384, 256),
130         std::make_pair(32768, 256),
131         std::make_pair(65536, 256),
132         std::make_pair(131072, 256),
133         std::make_pair(262144, 512),
134         std::make_pair(524288, 512),
135         std::make_pair(1048576, 1024)
136     };
137     runAllTests<testCases>();
138
139     std::cout << "-----" << std::endl;
140     std::cout << "Tests for constant n but different block sizes:" << std::endl;
141
142     constexpr std::array<std::pair<int, int>, 8> testCases2 = {
143         std::make_pair(1048576, 8),
144         std::make_pair(524288, 16),
145         std::make_pair(262144, 32),
146         std::make_pair(131072, 64),
147         std::make_pair(65536, 128),
148         std::make_pair(32768, 256),
149         std::make_pair(16384, 512),
150         std::make_pair(8192, 1024)
151     };
152     runAllTests<testCases2>();
153     return 0;
154 }
155
156
```

Funkcja dla cpu jest raczej prosta. Ciekawsza jest funkcja dla gpu. Obliczanie całki składa się z 3 etapów:

1. Równoległe policzenie wartości funkcji w punktach
2. Obliczanie sum częściowych dla poszczególnych bloków
3. Dodanie przez wątek nr 0 sumy w bloku do globalnej sumy

```

18
19 __global__ void gpuIntegral(double a, double b, int n, double *globalResult)
20 {
21     extern __shared__ double sdata[];
22
23     int tid = threadIdx.x;
24     int index = blockIdx.x * blockDim.x + threadIdx.x;
25     double h = (b - a) / n;
26
27     if (index < n) {
28         double x = a + (index + 0.5) * h;
29         sdata[tid] = h * d_function(x);
30     } else {
31         sdata[tid] = 0.0;
32     }
33     __syncthreads();
34
35     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
36         if (tid < s) {
37             sdata[tid] += sdata[tid + s];
38         }
39         __syncthreads();
40     }
41
42     if (tid == 0) {
43         atomicAdd(globalResult, sdata[0]);
44     }
45 }
46
47 __host__ double cpuIntegral(double a, double b, int n)
48 {
49     double h, sum = 0.0, x;
50     h = (b - a) / n;
51     for (int i = 0; i < n; i++) {
52         x = a + (i + 0.5) * h;
53         sum += h * function(x);
54     }
55     return h * sum;
56 }
57

```

2. Wykonanie

```

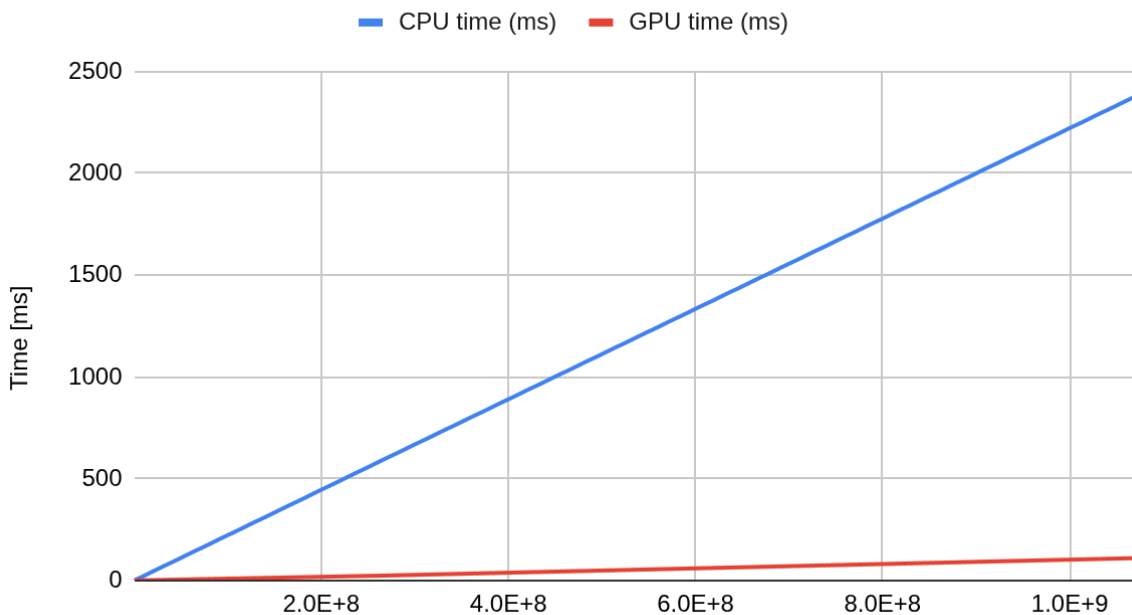
yonell@fedora:~/Infra/PK-PRIR-2025/lab7$ make run-Caliki
nvcc --std=c++20 -o Calki Calki.cu -arch=native
./Calki
Test Case: n: 256, numBlocks: 16, blockSize: 16, CPU time: 0.000640 ms, GPU time: 0.070489 ms, CPU result: 3.141594, GPU result: 3.141594
Test Case: n: 1024, numBlocks: 64, blockSize: 16, CPU time: 0.002340 ms, GPU time: 0.008010 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 4096, numBlocks: 256, blockSize: 16, CPU time: 0.009150 ms, GPU time: 0.012380 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8192, numBlocks: 256, blockSize: 32, CPU time: 0.010210 ms, GPU time: 0.006580 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 16384, numBlocks: 256, blockSize: 64, CPU time: 0.036229 ms, GPU time: 0.007190 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 65536, numBlocks: 1024, blockSize: 64, CPU time: 0.144769 ms, GPU time: 0.050760 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 262144, numBlocks: 4096, blockSize: 64, CPU time: 0.589603 ms, GPU time: 0.029449 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 524288, numBlocks: 4096, blockSize: 128, CPU time: 1.167426 ms, GPU time: 0.109769 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 1048576, numBlocks: 4096, blockSize: 256, CPU time: 2.347623 ms, GPU time: 0.101859 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 4194304, numBlocks: 16384, blockSize: 256, CPU time: 9.506468 ms, GPU time: 0.382665 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 32768, blockSize: 256, CPU time: 18.649191 ms, GPU time: 0.804741 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 16777216, numBlocks: 65536, blockSize: 256, CPU time: 37.293282 ms, GPU time: 1.546792 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 33554432, numBlocks: 131072, blockSize: 256, CPU time: 74.658933 ms, GPU time: 3.232662 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 134217728, numBlocks: 262144, blockSize: 512, CPU time: 298.558781 ms, GPU time: 12.198017 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 268435456, numBlocks: 524288, blockSize: 512, CPU time: 598.068890 ms, GPU time: 24.327754 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 1073741824, numBlocks: 1048576, blockSize: 1024, CPU time: 2385.153564 ms, GPU time: 110.196255 ms, CPU result: 3.141593, GPU result: 3.141593
-----
Tests for constant n but different block sizes:
Test Case: n: 8388608, numBlocks: 1048576, blockSize: 8, CPU time: 18.666361 ms, GPU time: 2.564610 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 524288, blockSize: 16, CPU time: 18.662981 ms, GPU time: 1.364293 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 262144, blockSize: 32, CPU time: 18.654131 ms, GPU time: 0.739771 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 131072, blockSize: 64, CPU time: 18.660311 ms, GPU time: 0.768101 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 65536, blockSize: 128, CPU time: 18.667631 ms, GPU time: 0.782601 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 32768, blockSize: 256, CPU time: 18.642040 ms, GPU time: 0.802440 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 16384, blockSize: 512, CPU time: 18.649971 ms, GPU time: 0.889550 ms, CPU result: 3.141593, GPU result: 3.141593
Test Case: n: 8388608, numBlocks: 8192, blockSize: 1024, CPU time: 18.640061 ms, GPU time: 0.901800 ms, CPU result: 3.141593, GPU result: 3.141593

```

3. Wykresy i omówienie wyników

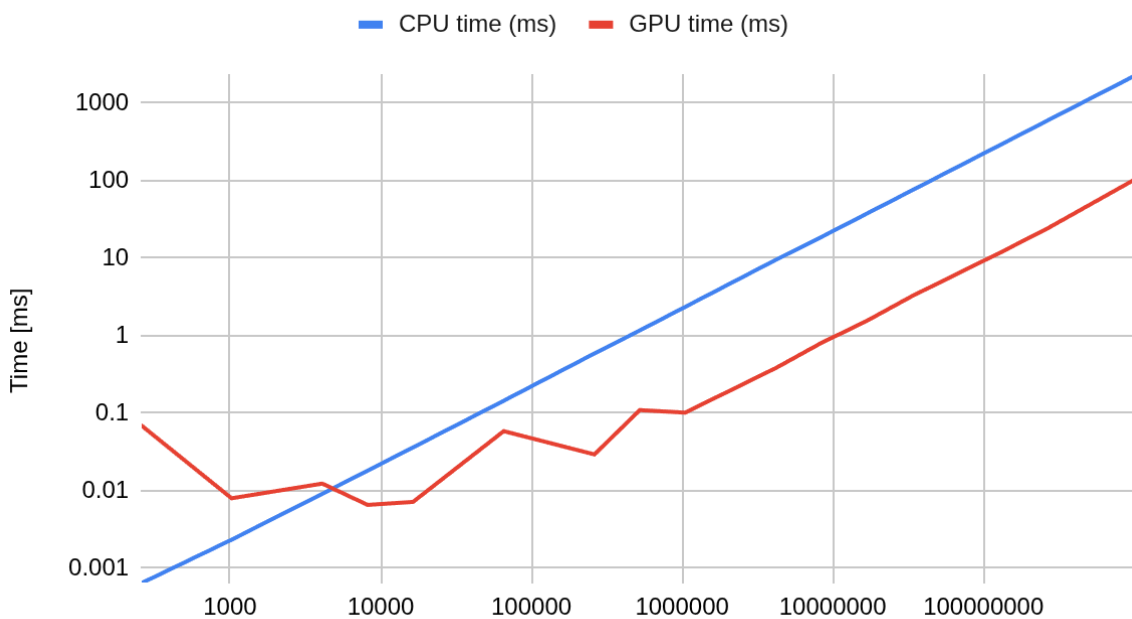
Wykres liniowy:

n, numBlocks, blockSize, CPU time (ms) i GPU time (ms)



Wykres logarytmiczny:

n, numBlocks, blockSize, CPU time (ms) i GPU time (ms)



Jak widzimy, pomimo tego, że gpu radzi sobie o wiele lepiej z całkowaniem, to dla dużych ilości przedziałów całkowania nadal ma złożoność liniową. Wydawało mi się, że wynika to z końcowego sumowania sum częściowych z bloków do sumy końcowej (etap 3), ale sprawdziłem jak ilość bloków wpływa na czas obliczeń i jest on w zasadzie stały. Nie wiem z czego może to wynikać. Raczej nie wynika to też z etapu 2, bo teoretycznie powinien on mieć złożoność logarytmiczną.